Big Data et bases de données NoSQL

Informatique — Licence 3 (L3)

Cours INF-L3-09

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours

Comprendre quand le modèle relationnel atteint ses limites et comment y répondre : les cinq V du Big Data, les familles de bases NoSQL (documents, clés-valeurs, colonnes), la modélisation avec MongoDB, le traitement distribué avec Hadoop et Spark, les entrepôts et lacs de données, et surtout la méthode pour choisir, cas par cas, entre SQL et NoSQL.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Les cinq V du Big Data et les limites du modèle relationnel
• Leçon 2 — Bases NoSQL : documents, clés-valeurs et familles de colonnes
• Leçon 3 — Modélisation NoSQL avec MongoDB
• Leçon 4 — Traitement distribué : concepts de Hadoop et de Spark
• Leçon 5 — Entrepôts de données et lacs de données
• Leçon 6 — Choisir entre SQL et NoSQL selon le besoin métier
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

Chaque jour, les applications que nous utilisons produisent des masses de données considérables : messages, photos, positions GPS, historiques d'achat, mesures de capteurs, journaux de serveurs. À partir d'un certain volume, et surtout quand ces données arrivent vite et sous des formes très variées, les outils classiques — une base relationnelle sur un seul serveur — montrent leurs limites. Le Big Data désigne à la fois ce phénomène et l'ensemble des techniques conçues pour stocker, traiter et analyser ces données à l'échelle de dizaines ou de milliers de machines.

Ce cours prolonge directement ceux de L2 sur SQL et la normalisation : loin d'opposer deux mondes ennemis, il montre que le NoSQL est né de problèmes précis que le relationnel ne savait pas résoudre seul, et que chaque famille de bases fait des compromis différents. Tu y apprendras le vocabulaire exact (CAP, BASE, sharding, réplication), la modélisation documentaire avec MongoDB, et les principes du traitement distribué avec Hadoop et Spark.

Le fil conducteur reste le choix raisonné : un bon informaticien ne choisit pas une technologie parce qu'elle est à la mode, mais parce qu'elle correspond aux accès réels de son application, à ses exigences de cohérence et à son budget. La dernière leçon est donc un atelier de décision, cas par cas, comme tu auras à en mener en entreprise.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Définir le Big Data par ses cinq V (volume, vélocité, variété, véracité, valeur) et illustrer chacun par un exemple concret.
- Expliquer les limites du modèle relationnel face aux très gros volumes et aux données hétérogènes.
- Distinguer les quatre familles NoSQL (clés-valeurs, documents, colonnes, graphes) et citer un usage type pour chacune.
- Modéliser une base documentaire MongoDB : imbriquer ou référencer, écrire des requêtes find et des pipelines d'agrégation simples.
- Énoncer correctement le théorème CAP et opposer les garanties ACID aux garanties BASE.
- Décrire l'architecture Hadoop (HDFS, MapReduce, YARN) et la différence fondamentale avec Spark (traitement en mémoire).
- Différencier un entrepôt de données d'un lac de données et justifier un choix SQL ou NoSQL selon le besoin métier.

Leçon 1 — Les cinq V du Big Data et les limites du modèle relationnel

On caractérise le Big Data par cinq propriétés, les « cinq V ». Le Volume d'abord : on ne parle plus en mégaoctets mais en téraoctets ou pétaoctets de données. La Vélocité ensuite : les données affluent en continu, comme les publications d'un réseau social ou les relevés d'un capteur, et il faut parfois réagir en quelques secondes. La Variété : aux tables bien rangées s'ajoutent textes libres, images, sons, vidéos, documents JSON ou XML dont la structure varie. La Véracité : ces données sont bruitées, incomplètes, parfois contradictoires ; leur qualité n'est pas garantie. Enfin la Valeur : tout ce volume ne sert à rien si l'on n'en extrait pas une information utile — une tendance, une alerte, une décision.

Le modèle relationnel, vu en L2, reste excellent pour les données structurées et les transactions rigoureuses. Mais il bute sur trois murs face au Big Data. Le mur du volume : une base relationnelle classique vit sur un serveur ; pour grandir, on achète une machine plus puissante (montée en charge verticale), ce qui devient vite très coûteux, puis impossible. Le mur de la structure : le schéma fixe et la normalisation obligent à découper chaque information dans des tables séparées, puis à multiplier les jointures pour reconstituer un objet complet — à très grande échelle, ces jointures deviennent lentes sur des milliards de lignes. Le mur de la disponibilité : répliquer une base relationnelle sur plusieurs continents en gardant une cohérence immédiate partout est un problème redoutable.

La réponse des géants du Web, au début des années 2000, a été double : répartir les données sur de nombreuses machines ordinaires (montée en charge horizontale, ou scaling horizontal) et assouplir le modèle — schéma flexible, jointures limitées, cohérence parfois relâchée. C'est de ce double mouvement qu'est né le NoSQL, littéralement « Not Only SQL » : non pas un rejet de SQL, mais un élargissement de la boîte à outils. Retiens dès maintenant que ces systèmes échangent une partie des garanties du relationnel contre la capacité de passer à l'échelle : toute la question, traitée dans ce cours, est de savoir ce que l'on gagne, ce que l'on perd, et si le

compromis convient à ton application.

L'essentiel
Big Data = Volume, Vélocité, Variété, Véracité, Valeur. Le relationnel atteint ses limites en volume (un seul serveur), en structure (schéma rigide, jointures coûteuses) et en disponibilité mondiale. La réponse NoSQL : distribution horizontale et compromis sur les garanties.

Exemple — Les cinq V appliqués à une plateforme de messagerie

```
Cas : une messagerie mobile utilisee par des millions de personnes

Volume : 40 To de messages et de photos archives chaque mois.
Velocite : 25 000 messages envoyes par seconde aux heures de pointe.
Variete : textes, photos, notes vocales, videos, positions GPS,
documents PDF : aucun format unique.
Veracite : doublons de messages apres reconnexion, profils
incomplets, horodatages incoherents entre telephones.
Valeur : detecter les sujets qui montent, mesurer l'audience
d'une campagne, reperer les comportements abusifs.

Diagnostic : le Volume et la Velocite depassent ce qu'un serveur
relationnel unique absorberait ; la Variete casse l'idee d'un
schema fixe unique. En revanche, la comptabilite de l'entreprise,
elle, reste petite, structuree et critique : elle restera en SQL.
```

À retenir

- Les cinq V : Volume, Vélocité, Variété, Véracité, Valeur — les citer et les illustrer fait partie des questions classiques d'examen.
- Montée en charge verticale = machine plus puissante ; horizontale = plus de machines. Le Big Data repose sur la seconde.
- NoSQL signifie « Not Only SQL » : un complément du relationnel, pas son remplacement systématique.

Exercice

Un hôpital veut numériser ses archives : 2 millions de dossiers patients, chacun composé de formulaires structurés (identité, constantes), de comptes rendus en texte libre, de radiographies (images) et de résultats d'analyses. Les archives sont consultées rarement mais doivent être conservées 30 ans. Identifie quels V du Big Data sont réellement présents dans ce cas, lesquels sont faibles, et dis si une base relationnelle classique suffirait pour la partie structurée.

✔ Corrigé
Volume : présent mais modéré — 2 millions de dossiers restent gérables, même si les images pèsent lourd (quelques dizaines de téraoctets au plus). Vélocité : faible, les archives grandissent lentement. Variété : forte — formulaires, textes libres et images cohabitent. Véracité : enjeu réel (vieux dossiers parfois incomplets). Valeur : surtout réglementaire et médicale en cas de réadmission. Conclusion : ce n'est pas un « vrai » cas Big Data au sens des plateformes géantes ; la partie structurée (identité, constantes, analyses) tient très bien dans une base relationnelle, et l'on stockera les images dans un système de fichiers ou un stockage objet, en ne gardant que leurs références en base. Le réflexe à garder : nommer les V un par un évite d'acheter une usine à gaz pour un problème simple.

Leçon 2 — Bases NoSQL : documents, clés-valeurs et familles de colonnes

Il n'existe pas UN modèle NoSQL mais plusieurs familles, chacune optimisée pour un type d'accès. Les bases clés-valeurs sont les plus simples : on stocke une valeur (nombre, texte, objet sérialisé) sous une clé unique, et l'on ne peut relire que par la clé exacte — comme un immense dictionnaire. Redis et DynamoDB en sont des exemples connus ; on s'en sert pour les caches, les sessions d'utilisateurs ou les compteurs, là où la rapidité d'une lecture par clé prime sur toute requête complexe.

Les bases documents, dont MongoDB est la représentante la plus connue, stockent des documents au format JSON (en pratique BSON) regroupés en collections. Un document peut contenir des sous-objets et des tableaux : toute une commande, avec son client et ses articles, tient en un seul document, là où le relationnel aurait exigé trois tables et deux jointures. Le schéma est flexible : deux documents d'une même collection peuvent avoir des champs différents, ce qui épouse bien les données hétérogènes. Les bases en familles de colonnes (Cassandra, HBase) organisent les données en lignes identifiées par une clé, mais où chaque ligne peut posséder ses propres colonnes regroupées en familles ; elles excellent pour écrire massivement des séries temporelles ou des journaux d'événements répartis sur beaucoup de machines. On ajoutera, pour être complet, les bases graphes (Neo4j), spécialisées dans les relations entre entités — amis d'amis, recommandations, détection de fraude.

Ces systèmes partagent un vocabulaire qu'il faut maîtriser. La réplication copie les données sur plusieurs machines pour survivre aux pannes et rapprocher les lectures des utilisateurs. Le partitionnement (sharding) découpe l'ensemble des données en morceaux répartis sur les machines : par exemple, les clients dont le nom commence par A–M sur un serveur, N–Z sur un autre. Mais qui dit distribution dit problèmes de coordination : si deux copies d'une même donnée divergent pendant une coupure réseau, laquelle fait foi ? C'est exactement la question du théorème CAP, formalisé par Eric Brewer : lors d'une partition du réseau (une coupure qui sépare les machines en groupes incapables de communiquer), un système distribué ne peut pas garantir à la fois la Cohérence (toute lecture reçoit la dernière écriture) et la Disponibilité (toute requête reçoit une réponse). Il doit choisir : refuser d'écrire en attendant que le réseau revienne (priorité cohérence), ou continuer à répondre en acceptant des copies

temporairement différentes (priorité disponibilité). La tolérance aux partitions n'est pas vraiment une option : dans un système réellement distribué, les coupures arrivent, on la subit. Voilà pourquoi on dit que les bases NoSQL privilégient souvent la disponibilité et une cohérence « éventuelle » (les copies finissent par converger), résumée par l'acronyme BASE — Basically Available (globalement disponible), Soft state (état pouvant évoluer même sans écriture), Eventually consistent (cohérent à terme) — là où le relationnel vise les propriétés ACID vues en L2 (Atomicité, Cohérence, Isolation, Durabilité) pour chaque transaction.

L'essentiel
Quatre familles NoSQL : clés-valeurs (lecture par clé, vitesse), documents (JSON flexible, objets entiers), familles de colonnes (écritures massives, séries temporelles), graphes (relations). Distribution = réplication + sharding. CAP : en cas de partition réseau, on choisit entre cohérence immédiate et disponibilité ; BASE (cohérence à terme) s'oppose en esprit à ACID.

Exemple — La même commande, en relationnel puis en document

```
-- Version relationnelle (3 tables, 2 jointures pour tout lire)
-- COMMANDE(id, date, statut, id_client)
-- CLIENT(id, nom, ville)
-- LIGNE_COMMANDE(id_commande, ref_produit, prix, qte)

-- Version documentaire (MongoDB) : UN document par commande
{
"_id": "cmd2026-001",
"client": { "nom": "Awa Diallo", "ville": "Bamako" },
"date": "2026-03-14",
"articles": [
{ "ref": "ORD-100", "prix": 45000, "qte": 1 },
{ "ref": "ECR-200", "prix": 25000, "qte": 2 }
],
"total": 95000,
"statut": "livree"
}

-- Lecture complete : 1 document, 0 jointure.
-- Contrepartie : le nom du client est recopie dans chaque
-- commande ; s'il change, il faut mettre a jour N documents.
```

À retenir

- Clés-valeurs = accès par clé uniquement ; documents = objets JSON complets ; colonnes = écritures massives ; graphes = relations.
- CAP, énoncé exact : pendant une partition réseau, impossible de garantir à la fois cohérence immédiate ET disponibilité — il faut choisir.
- ACID = garanties fortes par transaction ; BASE = disponibilité d'abord, cohérence obtenue « à terme » (éventuelle).

Exercice

Pour chacun de ces besoins, choisis une famille NoSQL (ou le relationnel) et justifie en une phrase : 1) stocker les sessions de connexion d'un site (clé = identifiant de session, lecture à chaque page, durée de vie 30 minutes) ; 2) catalogue de produits d'une boutique en ligne dont les fiches ont des caractéristiques très variables selon les catégories ; 3) relevés de compteurs électriques transmis toutes les minutes par 500 000 compteurs ; 4) la paie du personnel d'une PME.

✔ Corrigé
1) Clés-valeurs (type Redis) : accès uniquement par la clé de session, besoin de vitesse et d'expiration automatique — les requêtes complexes sont inutiles. 2) Documents (type MongoDB) : chaque fiche produit a ses propres champs (taille d'écran pour un téléphone, pointure pour une chaussure) ; le schéma flexible et la lecture d'une fiche entière en un document sont idéaux. 3) Familles de colonnes (type Cassandra) : flux d'écritures massives et régulières, données naturellement organisées par compteur et par date, tolérance à une cohérence relâchée. 4) Relationnel, sans hésiter : volumes modestes, données très structurées et exigences ACID absolues (un salaire ne se « converge » pas à terme, il doit être exact immédiatement).

Leçon 3 — Modélisation NoSQL avec MongoDB

Modéliser en MongoDB ne consiste pas à transposer ses tables : le point de départ n'est plus la structure des données, mais les requêtes de l'application. La question directrice est : « quelles informations l'écran affiche-t-il ensemble, et combien de fois par seconde ? » Les données lues ensemble gagnent à être stockées ensemble, dans le même document — c'est le principe de l'imbrication (embedding). À l'inverse, les données partagées par beaucoup de documents, mises à jour souvent, ou dont le nombre croît sans limite, seront plutôt référencées : le document stocke alors l'identifiant de l'autre, et l'application fait deux lectures — c'est le principe de la référence, l'équivalent lointain de la clé étrangère.

Imbriquer a un prix qu'il faut connaître : la duplication. Si le nom et la ville du client vivent dans chaque commande, une correction de la ville doit être appliquée à toutes les commandes passées — ou bien l'on décide que la commande garde volontairement la ville du moment de l'achat (une « photographie »), ce qui est souvent le bon choix métier. MongoDB impose aussi une limite de taille de 16 Mo par document : imbriquer une liste qui grandit indéfiniment, comme tous les commentaires d'un article très populaire, finit par heurter ce plafond ; on la déplacera alors dans sa propre collection, référencée par l'identifiant de l'article. Règle pratique : imbriquer ce qui est lu ensemble, borné en taille et rarement modifié séparément ; référencer le reste.

Côté requêtes, le shell mongosh parle en objets JavaScript. db.commandes.find({ ville: "Bamako" }) filtre les documents, .sort(), .limit() ordonnent et bornent, et le cadre d'agrégation enchaîne des étapes (pipeline) : $match filtre, $group regroupe et calcule (sommes, moyennes), $sort trie, $project choisit les champs. C'est l'équivalent du SELECT … WHERE … GROUP BY de SQL, décomposé en étapes lisibles. Enfin, comme en relationnel, les performances viennent des index : createIndex crée des structures qui évitent de parcourir toute la collection à chaque recherche — sans index, MongoDB lit tous les documents, exactement comme un SELECT sans index en SQL.

L'essentiel
En MongoDB, on modélise à partir des requêtes, pas des tables : imbriquer ce qui est lu ensemble et borné (attention à la limite de 16 Mo par document), référencer ce qui est partagé, mis à jour souvent ou illimité. Requêtes via find et les pipelines d'agrégation ($match, $group, $sort) ; les index restent indispensables.

Exemple — Requêtes mongosh sur la collection « commandes »

```
// Insertion d'un document (remarquer le champ supplementaire
// « code_promo », absent de la commande precedente : le
// schema flexible l'accepte sans migration)
db.commandes.insertOne({
_id: "cmd2026-002",
client: { nom: "Moussa Keita", ville: "Segou",
telephone: "+223 70 00 00 00" },
date: "2026-03-15",
articles: [ { ref: "CLA-300", prix: 8000, qte: 3 } ],
total: 24000,
statut: "en cours",
code_promo: "IAM10"
})

// Lecture filtree : les commandes livrees
// (equivalent SQL : WHERE statut = 'livree')
db.commandes.find({ statut: "livree" })

// Lecture dans un sous-document : ville du client
db.commandes.find({ "client.ville": "Bamako" })

// Agregation : chiffre d'affaires livre, par ville
// ($match = WHERE, $group = GROUP BY + SUM)
db.commandes.aggregate([
{ $match: { statut: "livree" } },
{ $group: { _id: "$client.ville",
ca: { $sum: "$total" } } },
{ $sort: { ca: -1 } }
])
// Resultat avec nos deux commandes : [ { _id: "Bamako", ca: 95000 } ]

// Index pour accelerer la recherche par statut
db.commandes.createIndex({ statut: 1 })
```

À retenir

- On modélise à partir des accès de l'application : les données lues ensemble se stockent ensemble.
- Imbrication = lecture rapide mais duplication ; référence = pas de duplication mais lectures multiples. Il faut arbitrer, pas appliquer une règle aveugle.
- Pipeline d'agrégation : $match filtre, $group regroupe et calcule, $sort trie — l'héritier du GROUP BY de SQL.

Exercice

Tu dois modéliser, pour une plateforme de cours en ligne, des « cours » (titre, enseignant, prix) et leurs « avis » laissés par les étudiants (note, commentaire, date). Un cours peut recevoir des milliers d'avis, affichés par pages de 20 ; la fiche d'un cours affiche seulement sa note moyenne. Faut-il imbriquer les avis dans le document du cours ou les référencer dans une collection séparée ? Justifie avec au moins deux arguments tirés de la leçon, et dis où tu stockerais la note moyenne.

✔ Corrigé
Les avis doivent être référencés, dans leur propre collection (avis : { id_cours, note, commentaire, date }). Premier argument : le nombre d'avis est illimité — imbriqués, ils gonfleraient le document du cours jusqu'à la limite de 16 Mo et rendraient chaque lecture de fiche plus lourde, alors qu'on n'affiche que 20 avis par page. Deuxième argument : les avis vivent leur propre vie (ajout, modération, suppression) indépendamment de la fiche du cours. En revanche, la note moyenne est lue à chaque affichage de la fiche : on la stocke dénormalisée dans le document du cours (avec le nombre d'avis), recalculée ou ajustée à chaque nouvel avis — c'est une duplication assumée et rentable, exactement le compromis que la modélisation documentaire demande de peser à chaque champ.

Leçon 4 — Traitement distribué : concepts de Hadoop et de Spark

Stocker des pétaoctets ne suffit pas : il faut aussi pouvoir les traiter. Le principe du traitement distribué est de déplacer le calcul vers les données, plutôt que de rapatrier les données vers un calculateur unique : on découpe le travail en tâches indépendantes que des dizaines de machines exécutent en parallèle sur les morceaux qu'elles détiennent déjà. Apache Hadoop, né chez Yahoo à partir de travaux publiés par Google, est le socle historique de cette approche. Il repose sur trois briques : HDFS, le système de fichiers distribué ; MapReduce, le modèle de calcul ; et YARN, le gestionnaire qui attribue les ressources (processeur, mémoire) des machines aux tâches qui en demandent.

HDFS découpe chaque fichier en gros blocs (128 Mo par défaut) répartis sur les machines du cluster, chaque bloc étant répliqué — trois fois par défaut — sur des machines différentes : la panne d'un disque ou d'un serveur ne perd donc aucune donnée. Un maître, le NameNode, conserve l'annuaire (quel fichier, quels blocs, sur quelles machines) pendant que les DataNodes stockent les blocs. MapReduce, lui, organise le calcul en deux temps : le Map transforme chaque élément d'entrée en paires (clé, valeur) ; une phase de tri et de regroupement (shuffle) rassemble toutes les valeurs d'une même clé ; puis le Reduce combine ces valeurs en un résultat par clé. Compter les mots d'un corpus gigantesque est l'exemple canonique : le Map émet (mot, 1) pour chaque mot lu, le Reduce additionne les 1 de chaque mot.

MapReduce écrit ses résultats intermédiaires sur disque entre chaque étape : fiable, mais lent pour les algorithmes qui enchaînent beaucoup d'étapes, comme l'apprentissage automatique. Apache Spark a été conçu pour lever cette limite : il conserve les données intermédiaires en mémoire vive des machines (structures appelées RDD, puis DataFrames) et peut ainsi enchaîner les transformations beaucoup

plus vite — on cite souvent des gains de vitesse considérables sur les traitements itératifs. Spark n'est pas un rival total d'Hadoop : il cohabite avec lui, en lisant les fichiers sur HDFS et en demandant ses ressources à YARN. Retiens la division du travail : Hadoop = le socle (stockage HDFS, ressources YARN, calcul historique MapReduce) ; Spark = un moteur de calcul plus rapide, surtout en mémoire.

L'essentiel
Traitement distribué = déplacer le calcul vers les données et paralléliser. Hadoop : HDFS (stockage en blocs répliqués, NameNode et DataNodes), MapReduce (Map → regroupement → Reduce), YARN (gestion des ressources). Spark : même idée, mais étapes intermédiaires gardées en mémoire, donc bien plus rapide sur les calculs itératifs ; il s'appuie souvent sur HDFS et YARN.

Exemple — MapReduce pas à pas : compter les mots d'une phrase

```
Phrase traitee (15 mots) :
"le big data change le monde et le monde change
les donnees du big data"

ETAPE MAP : chaque mot devient une paire (mot, 1)
(le,1) (big,1) (data,1) (change,1) (le,1) (monde,1)
(et,1) (le,1) (monde,1) (change,1) (les,1)
(donnees,1) (du,1) (big,1) (data,1)
-> 15 paires emises

ETAPE SHUFFLE : regroupement par cle
big : [1, 1]
change : [1, 1]
data : [1, 1]
donnees : [1]
du : [1]
et : [1]
le : [1, 1, 1]
les : [1]
monde : [1, 1]

ETAPE REDUCE : somme des valeurs de chaque cle
big = 2 change = 2 data = 2 donnees = 1
du = 1 et = 1 le = 3 les = 1
monde = 2
Controle : 2+2+2+1+1+1+3+1+2 = 15 = nombre de mots.

Sur un vrai cluster, des milliers de Map traitent chacun
un bloc du fichier, puis les Reduce additionnent par mot :
le programme ne change pas, seule l'echelle change.
```

À retenir

- HDFS = stockage distribué en blocs répliqués (3 fois par défaut) ; NameNode = l'annuaire, DataNodes = les blocs.

• MapReduce : Map produit des paires (clé, valeur), le shuffle regroupe par clé, Reduce combine. Simple et robuste, mais avec écritures disque entre étapes.

• Spark = calcul en mémoire sur RDD/DataFrames : plus rapide, notamment pour les traitements itératifs ; il utilise souvent HDFS et YARN d'Hadoop.

Exercice

Un fichier de journaux (logs) d'un site web contient des millions de lignes du type « 2026-03-14 10:02 GET /accueil 200 ». On veut le nombre de requêtes par code de statut HTTP (200, 404, 500…). Décris le traitement en MapReduce : que doit émettre le Map pour chaque ligne, que reçoit le Reduce, et que calcule-t-il ? Puis explique en deux phrases pourquoi, si l'on devait ensuite analyser ces mêmes logs vingt fois de suite pour un modèle statistique, Spark serait préférable à MapReduce classique.

✔ Corrigé

Map : pour chaque ligne lue, extraire le code de statut et émettre la paire (code, 1) — par exemple (200, 1), (404, 1)… Shuffle : le système regroupe les listes de 1 par code : 200 → [1, 1, 1, …], 404 → [1, 1, …]. Reduce : pour chaque code, additionner les 1 de sa liste ; le résultat est le nombre total de requêtes par statut. Quant à Spark : MapReduce réécrit ses résultats sur disque à chaque étape, donc vingt analyses successives paieraient vingt fois les lectures et écritures disque ; Spark charge les logs une fois en mémoire (DataFrames) et enchaîne les vingt transformations sans repasser par le disque, ce qui est précisément son avantage sur les traitements répétés et itératifs.

Leçon 5 — Entrepôts de données et lacs de données

Les bases des leçons précédentes servent les applications au quotidien (on parle de traitement transactionnel : enregistrer une commande, lire une fiche). L'analyse décisionnelle pose un autre problème : croiser des années d'historique, toutes activités confondues, pour répondre à des questions comme « quelles catégories de produits progressent dans chaque région, trimestre par trimestre ? ». Interroger directement la base de production ralentirait les ventes et mêlerait des données brutes hétérogènes ; on construit donc un système séparé, alimenté périodiquement par extraction depuis les sources.

L'entrepôt de données (data warehouse) est ce système classique : il reçoit des données nettoyées, transformées et structurées à l'avance selon un schéma pensé pour l'analyse (« schéma à l'écriture »). Le modèle le plus répandu y est le schéma en étoile : au centre, une table des faits contient les événements mesurés (chaque ligne de vente, avec ses montants et quantités) ; autour, des tables de dimensions décrivent les axes d'analyse (le temps, le produit, le magasin, le client), volontairement dénormalisées pour permettre des regroupements rapides. Des outils de restitution (tableaux de bord, requêtes d'agrégation) branchés sur l'entrepôt donnent alors des temps de réponse réguliers, car le travail de nettoyage et de jointure a été fait une fois pour toutes à l'alimentation.

Le lac de données (data lake) inverse la philosophie : on y verse les données brutes, dans leur format d'origine — tables, mais aussi textes, images, journaux, fichiers JSON — sur un stockage peu coûteux (souvent HDFS ou un stockage objet dans le cloud), et l'on ne décide de leur structure qu'au moment de les exploiter (« schéma à la lecture »). Le lac est précieux pour conserver ce dont on ne connaît pas encore l'usage, et il alimente volontiers les projets de data science (vue en L2). Mais sans discipline — catalogue des contenus, contrôle d'accès, gestion de la qualité — il dégénère en « marécage de données » (data swamp) où plus personne ne retrouve ni ne fait confiance à rien. En pratique, beaucoup d'organisations combinent les deux : le lac reçoit et conserve tout, l'entrepôt expose les données propres et gouvernées aux décideurs. On parle aussi de maisons de lac (lakehouse), architectures récentes qui cherchent à réunir les deux approches sur un même stockage.

L'essentiel
Entrepôt = données propres, structurées à l'avance (schéma à l'écriture), optimisées pour l'analyse décisionnelle, souvent en schéma en étoile (faits + dimensions). Lac = données brutes conservées en formats d'origine, structurées à la lecture ; flexible et bon marché, mais devient un « marécage » sans gouvernance. Les deux sont complémentaires.

Exemple — Schéma en étoile d'un entrepôt de ventes

```
+------------------+
| DIM_TEMPS |
| id_date (PK) |
| jour, mois, |
| trimestre, annee|
+--------+---------+
|
+---------------+ +----+-------------+ +---------------+
| DIM_PRODUIT | | FAIT_VENTES | | DIM_MAGASIN |
| id_produit +---+ id_date (FK) +---+ id_magasin |
| nom, | | id_produit (FK) | | ville, |
| categorie, | | id_magasin (FK) | | region |
| marque | | id_client (FK) | +---------------+
+---------------+ | quantite |
| montant |
+---+ remise |
| +------------------+
|
+------+-------+
| DIM_CLIENT |
| id_client |
| segment, |
| ville |
+--------------+

Lecture : chaque ligne de FAIT_VENTES est une vente mesuree
(quantite, montant) ; les dimensions donnent les axes
d'analyse. Question type, resolue par une agregation :
« montant total par categorie de produit et par trimestre »
= SUM(montant) groupe par DIM_PRODUIT.categorie
et DIM_TEMPS.trimestre. Aucune jointure vers la base de
production : l'entrepot est autonome et dimensionne pour cela.
```

À retenir

- Schéma en étoile : une table des faits (mesures) entourée de tables de dimensions (axes d'analyse).
- Entrepôt = schéma à l'écriture (propre, décidé d'avance) ; lac = schéma à la lecture (brut, décidé à l'usage).
- Un lac sans catalogue ni règles de qualité devient un marécage de données inutilisable.

Exercice

Une université conserve : (a) les notes officielles et les inscriptions, exploitées chaque semestre par la direction ; (b) les journaux de connexion de sa plateforme de cours (des millions de lignes brutes) ; (c) les enregistrements audio des cours magistraux, dont on envisage un jour la transcription automatique. Pour chaque ensemble, choisis entrepôt ou lac et justifie en une phrase. Que faut-il mettre en place dès le départ pour que le lac ne devienne pas un marécage ?

✔ Corrigé
(a) Entrepôt : données structurées, usage décisionnel récurrent et connu, besoin de chiffres fiables et comparables d'un semestre à l'autre. (b) Lac : volume énorme, format brut, usages encore exploratoires (statistiques d'usage, recherche) — les structurer tous à l'avance coûterait cher pour un bénéfice incertain. (c) Lac également : fichiers audio bruts conservés pour un traitement futur (la transcription) dont on ne connaît pas encore la forme exacte. Pour éviter le marécage : tenir un catalogue (qui a déposé quoi, quand, dans quel format), appliquer des règles de nommage et d'accès, dater et documenter chaque jeu de données, et prévoir une politique de qualité et de suppression. Un lac se gouverne dès le premier fichier, pas après le millième.

Leçon 6 — Choisir entre SQL et NoSQL selon le besoin métier

Après ce parcours, la tentation serait de couronner un vainqueur. Ce serait une erreur : le choix d'une base de données est un arbitrage entre des exigences concrètes — nature des données, requêtes dominantes, exigence de cohérence, volume et croissance, compétences de l'équipe. Le relationnel reste le choix par défaut dès que les données sont structurées, que les relations entre entités sont nombreuses et interrogées dans tous les sens, et que les transactions doivent être exactes immédiatement : comptabilité, gestion de stocks, inscriptions, réservations. Ses cinquante ans de maturité, le langage SQL partagé par tous et les garanties ACID sont des atouts mesurables, pas des vieilleries.

Le NoSQL s'impose quand un ou plusieurs signaux forts apparaissent : volume ou débit qui dépasse le serveur unique ; données dont la structure varie ou évolue vite (catalogues, contenus, profils) ; accès dominés par la lecture d'objets entiers par clé plutôt que par des jointures ; besoin de disponibilité mondiale en acceptant une cohérence à terme. La méthode de décision tient en quatre questions. Premièrement : mes données ont-elles une structure stable et fortement relationnelle ? Deuxièmement : quelles sont les trois requêtes les plus fréquentes de l'application, et le modèle envisagé y répond-il simplement ? Troisièmement : que se passe-t-il métier si une lecture renvoie une donnée vieille de quelques secondes — anodin, gênant ou catastrophique ? Quatrièmement : quel volume dans trois ans, et l'équipe sait-elle exploiter cette technologie ? Ces réponses, documentées, valent mieux que n'importe quel effet de mode.

Enfin, la pratique réelle des entreprises est souvent mixte : on parle de persistance polyglotte. Une même plateforme peut garder ses paiements en relationnel, son catalogue en documents, ses sessions en clés-valeurs et ses journaux dans un lac, chaque donnée logée dans le système adapté à son usage — quitte à accepter la complexité d'administrer plusieurs technologies. La compétence attendue d'un licence 3 n'est donc pas de militer pour un camp, mais de justifier un choix, cas par cas, avec des arguments de volume, de cohérence, de requêtes et de coût. C'est exactement ce qu'entraînera le TD qui suit ce chapitre.

L'essentiel
Pas de vainqueur universel : le relationnel reste la référence pour données structurées, requêtes relationnelles variées et cohérence immédiate (ACID) ; le NoSQL répond aux volumes massifs, aux schémas flexibles et aux accès par objets entiers. Méthode : 4 questions (structure, requêtes dominantes, tolérance à la fraîcheur, volume futur et compétences). En pratique, la persistance est souvent polyglotte.

Exemple — Tableau de décision : cinq cas, cinq verdicts argumentés

```
CAS | VERDICT | ARGUMENT DECISIF
--------------------------+-------------+----------------------
1. Comptabilite d'une | SQL | Transactions ACID
PME, 200 ecritures/j | (relationnel)| obligatoires, petit
| | volume, rapports libres
--------------------------+-------------+----------------------
2. Catalogue e-commerce, | NoSQL | Fiches a champs tres
fiches tres variables, | documents | variables, lecture
2 millions de fiches | (MongoDB) | d'objets entiers
--------------------------+-------------+----------------------
3. Sessions et paniers | NoSQL | Acces par cle seule,
temporaires d'un site | cles-valeurs| expiration courte,
a fort trafic | (Redis) | vitesse critique
--------------------------+-------------+----------------------
4. Reseau social : | NoSQL | La question centrale
« amis d'amis », | graphes | est la relation elle-
recommandations | (Neo4j) | meme, pas les objets
--------------------------+-------------+----------------------
5. Historique de ventes | SQL | Donnees deja propres
pour rapports de la | (entrepot | et structurees,
direction | en etoile) | agregations variees
--------------------------+-------------+----------------------
Regle de lecture : aucun verdict ne vient de la mode ;
chacun cite l'exigence qui le decide (coherence, forme
des requetes, variete du schema, nature des acces).
```

À retenir

- La cohérence immédiate non négociable (argent, stocks critiques) désigne le relationnel ACID presque toujours.
- Les trois requêtes les plus fréquentes de l'application sont le meilleur test d'un modèle de données.
- Persistance polyglotte : plusieurs bases dans un même système, chacune pour l'usage auquel elle excelle — un choix d'architecture, pas une incohérence.

Exercice

Un service de livraison de repas veut sa plateforme : restaurants et menus (fiches très variables selon les enseignes), commandes et paiements (argent réel), suivi géographique des livreurs en temps réel (une position toutes les 10 secondes), et statistiques mensuelles pour les restaurants partenaires. Propose une architecture de persistance argumentée : pour chaque type de donnée, le système choisi et

l'argument décisif.

✔ Corrigé
Menus et restaurants → base documents (MongoDB) : fiches hétérogènes (un menu n'a pas les mêmes champs qu'une fiche d'enseigne), lues entières à chaque consultation. Commandes et paiements → base relationnelle ACID : l'argent exige des transactions exactes immédiatement (pas de double débit « convergent à terme »), volumes maîtrisés. Positions des livreurs → double réponse : le flux en temps réel passe par un système clés-valeurs ou colonnes capable d'absorber des écritures massives et fréquentes avec une cohérence relâchée (une position vieille de 10 secondes est acceptable), et l'historique est déversé dans un lac pour analyses ultérieures. Statistiques mensuelles → entrepôt de données alimenté depuis les sources, en schéma en étoile (faits = lignes de commande, dimensions = temps, restaurant, ville), pour offrir aux partenaires des rapports rapides sans charger les bases de production. L'ensemble illustre la persistance polyglotte : quatre besoins, quatre compromis différents, chacun justifié.

Travaux dirigés (TD)

TD 1 — Modéliser le catalogue d'une librairie en MongoDB

Une librairie en ligne vend des livres (titre, auteur, prix, stock), des livres numériques (mêmes informations + format de fichier et taille) et des revues (titre, numéro, mois de parution, prix). L'écran principal affiche la fiche complète d'un article ; la recherche se fait par titre ou par auteur ; le stock est décrémenté à chaque vente et ne doit jamais passer en négatif. 1) Propose une collection « articles » et écris le document d'un livre papier et celui d'un livre numérique. 2) Le stock doit-il vivre dans le même document ? Discute le risque de cohérence en cas de ventes simultanées. 3) Écris la requête mongosh qui affiche les livres d'un auteur donné, triés par prix croissant. 4) Cite un cas où tu préférerais pourtant une base relationnelle pour cette librairie.

✔ Corrigé du TD 1
1) Une seule collection « articles » avec un champ « type » : le livre papier aura { _id, type: "papier", titre, auteur, prix, stock, pages } ; le livre numérique { _id, type: "numerique", titre, auteur, prix, format: "EPUB", taille_mo: 2.4 } — le schéma flexible accueille les champs propres à chaque type sans table d'héritage compliquée. 2) Le stock peut vivre dans le document (lecture de fiche instantanée), mais deux ventes simultanées du dernier exemplaire peuvent toutes deux lire stock = 1 et décider de vendre : MongoDB garantit l'atomicité des mises à jour d'un document unique, donc on utilisera un updateOne conditionnel ({ stock: { $gt: 0 } } avec $inc: { stock: -1 }) et on vérifiera qu'exactement un document a été modifié — le second acheteur échouera proprement. L'opération reste sûre, à condition de l'écrire ainsi plutôt que de lire puis réécrire. 3) db.articles.find({ auteur: "…", type: "papier" }).sort({ prix: 1 }) — en précisant le nom d'auteur du cas traité. 4) Si la librairie doit surtout produire des rapports croisés complexes (ventes par auteur, éditeur, période, librairie partenaire, avec des jointures imprévues et changeantes), ou brancher un logiciel de comptabilité classique, le relationnel redevient supérieur : SQL y est imbattable et les volumes d'une librairie ne justifient pas à eux seuls le NoSQL.

TD 2 — CAP et cohérence : trois incidents à analyser

Trois situations réelles simplifiées. Pour chacune, dis quel choix CAP le système a manifestement fait (priorité Cohérence ou priorité Disponibilité pendant une partition), et si ce choix te semble adapté au métier. A) Pendant une coupure réseau entre ses deux centres de données, une banque refuse temporairement les retraits à l'étranger plutôt que de risquer un solde faux. B) Pendant le même type de coupure, un réseau social continue d'accepter les publications ; certains utilisateurs ne voient les messages de leurs amis qu'une heure plus tard, dans le désordre. C) Un site marchand accepte des commandes pendant la coupure : deux clients achètent le dernier exemplaire d'un produit, et l'entreprise doit ensuite annuler une des deux commandes avec un geste commercial.

✔ Corrigé du TD 2

A) Choix Cohérence (CP) : le système préfère refuser le service (indisponibilité partielle) plutôt que de répondre avec des données potentiellement fausses. Adapté : pour un solde bancaire, une erreur de cohérence coûte infiniment plus cher qu'un refus temporaire. B) Choix Disponibilité (AP) : le service répond toujours, la cohérence devient éventuelle — les messages finissent par arriver partout. Adapté : un message vu une heure plus tard est un désagrément, pas un dommage ; bloquer toute publication serait pire pour le métier. C) Choix Disponibilité également, mais ici discutable : la disponibilité a été préservée au prix d'une incohérence métier réelle (survente du stock) qu'il a fallu compenser humainement et commercialement. La leçon : le choix CAP n'est pas qu'une affaire technique — il engage un coût métier, et pour un stock limité vendu au prix fort, une solution hybride (réservation stricte du stock en système cohérent, catalogue en système disponible) aurait été préférable. Ces trois cas montrent pourquoi l'énoncé exact de CAP parle de choix pendant une partition, et non de propriétés permanentes dans l'absolu.

Travaux pratiques (TP)

TP 1 — Prise en main de MongoDB et mini-MapReduce

Sur une machine de TP disposant de mongosh (ou d'une instance MongoDB Atlas gratuite), réalise la chaîne complète suivante. 1) Crée une base « biblio » et une collection « livres » ; insère au moins six documents de livres de ta conception (titre, auteur, annee, prix, genres sous forme de tableau). 2) Écris trois requêtes find : les livres d'un genre donné ; les livres parus après 2015 triés par prix décroissant ; les livres dont le prix est inférieur ou égal à 5 000. 3) Écris un pipeline d'agrégation donnant le prix moyen par genre (pense au rôle de $unwind sur le tableau des genres). 4) Ajoute un index sur le champ « auteur » et compare, avec explain(), le nombre de documents examinés avant et après (champ totalDocsExamined). 5) Hors de MongoDB, dans le langage de ton choix, implémente le MapReduce de comptage de mots de la leçon 4 sur un paragraphe d'une dizaine de mots que tu rédigeras, et vérifie que la somme des comptes égale le nombre de mots du paragraphe. Rédige un compte rendu d'une page : requêtes, résultats obtenus, et ce que l'index a changé.

✔ Attendus du TP 1
Attendus indicatifs (les valeurs dépendent de tes données) : les find utilisent des filtres d'égalité, { annee: { $gt: 2015 } } avec .sort({ prix: -1 }), et { prix: { $lte: 5000 } }. Le pipeline d'agrégation attendu a la forme : [{ $unwind: "$genres" }, { $group: { _id: "$genres", prix_moyen: { $avg: "$prix" } } }, { $sort: { prix_moyen: -1 } }] — $unwind déplie le tableau des genres pour qu'un livre multi-genres compte dans chacun, point que les étudiants oublient le plus souvent. Avant index, explain() montre un COLLSCAN avec totalDocsExamined égal au nombre total de documents ; après createIndex({ auteur: 1 }), la même recherche par auteur montre un IXSCAN et un nombre de documents examinés proche du nombre de résultats. Pour le MapReduce local, le contrôle décisif est la conservation du total : la somme des comptes par mot doit égaler le nombre de mots du paragraphe (15 dans l'exemple du cours) ; tout écart signale une erreur dans l'étape de regroupement. Le compte rendu est réussi s'il distingue clairement ce qui relève du modèle documentaire (requêtes, agrégations) et ce qui relève du traitement distribué (MapReduce), et s'il conclut sur l'effet mesuré de l'index plutôt que sur une impression.

Exercices corrigés supplémentaires

Exercice 1

Vrai ou faux, corrige les affirmations fausses. 1) « Le théorème CAP dit qu'un système distribué ne peut avoir qu'une seule des trois propriétés à la fois. » 2) « NoSQL signifie que le langage SQL est interdit dans ces systèmes. » 3) « Dans un lac de données, les données sont nettoyées avant d'y être versées. » 4) « HDFS stocke chaque fichier en un seul exemplaire pour économiser l'espace. »

✔ Corrigé
1) Faux : CAP concerne le comportement pendant une partition réseau ; dans ce cas, le système doit choisir entre cohérence immédiate et disponibilité (la tolérance aux partitions étant de fait imposée à tout système distribué). Sans partition, un système peut offrir les deux. 2) Faux : NoSQL signifie « Not Only SQL » ; certains systèmes NoSQL proposent d'ailleurs des langages de requête inspirés de SQL. 3) Faux : c'est le principe inverse — le lac reçoit les données brutes (schéma à la lecture) ; le nettoyage préalable caractérise l'entrepôt (schéma à l'écriture). 4) Faux : HDFS réplique les blocs, trois fois par défaut, précisément pour survivre aux pannes de machines.

Exercice 2

Une application de suivi d'entraînement sportif enregistre pour chaque utilisateur : son profil (nom, âge, ville), ses séances (date, type d'activité, durée, distance), et un classement hebdomadaire entre amis calculé la nuit. Les séances arrivent en continu depuis des montres connectées. Propose un stockage pour chaque type de donnée et identifie le point où une cohérence éventuelle est acceptable.

✔ Corrigé
Profils : base documents ou relationnelle — petit volume, lecture d'objet entier à chaque connexion ; les deux se défendent, le document simplifie l'évolution du profil. Séances : flux continu d'écritures, données datées et volumineuses à terme → familles de colonnes (type Cassandra) ou base documents partitionnée par utilisateur ; l'accès dominant est « les séances d'un utilisateur par date », ce qui guide la clé de partition. Classement hebdomadaire : calcul batch nocturne (pourquoi pas Spark sur les séances exportées) dont le résultat est stocké prêt à lire (clés-valeurs ou documents) puisqu'il n'est recalculé qu'une fois par semaine. Cohérence éventuelle acceptable : le classement et les statistiques d'amis — afficher un classement vieux de quelques minutes, voire de la veille, ne nuit à personne ; en revanche, l'enregistrement d'une séance ne doit pas être perdu : c'est la durabilité des écritures qui compte ici, plus que la fraîcheur des lectures sociales.

Exercice 3

On te donne ces étapes mélangées d'un pipeline Big Data : (i) les capteurs envoient leurs mesures ; (ii) un calcul Spark produit des moyennes horaires ; (iii) les mesures brutes sont conservées sur HDFS ; (iv) un tableau de bord affiche les moyennes ; (v) YARN attribue des ressources au calcul. Remets-les dans l'ordre du flux de données, nomme le rôle de chaque composant cité, et dis dans quel type de système (lac, entrepôt, base opérationnelle) les moyennes horaires ont vocation à être exposées aux décideurs si elles doivent être croisées avec d'autres indicateurs propres.

✔ Corrigé
Ordre : (i) émission par les capteurs → (iii) conservation des bruts sur HDFS → (v) YARN attribue les ressources → (ii) Spark calcule les moyennes horaires → (iv) restitution au tableau de bord. Rôles : HDFS = stockage distribué et répliqué des données brutes (c'est le socle du lac) ; YARN = gestionnaire de ressources du cluster, il décide quelles machines exécutent le calcul et avec combien de mémoire/processeur ; Spark = moteur de calcul en mémoire qui exécute le traitement. Si les moyennes doivent être croisées avec d'autres indicateurs propres et gouvernés, elles ont vocation à être exposées dans un entrepôt de données, alimenté depuis le lac : le lac conserve le brut, l'entrepôt publie le propre — c'est la combinaison la plus fréquente en entreprise.

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. Parmi les cinq V du Big Data, celui qui désigne la fiabilité et la qualité des données est…

A) La Vélocité

B) La Véracité

C) La Valeur

2. La montée en charge horizontale consiste à…

A) Acheter un serveur plus puissant

B) Éteindre les machines inutilisées

C) Ajouter des machines au système

3. Un document MongoDB correspond le mieux, en relationnel, à…

A) Une base entière

B) Une ligne (un enregistrement), éventuellement riche de sous-objets

C) Une colonne

4. Énoncé correct du théorème CAP : pendant une partition réseau, un système doit choisir entre…

A) Vitesse et prix

B) Cohérence immédiate et disponibilité

C) SQL et NoSQL

5. L'acronyme BASE désigne des systèmes qui privilégient…

A) Une cohérence immédiate stricte

B) Des transactions multi-tables

C) La disponibilité, avec une cohérence obtenue à terme

6. Dans Hadoop, le NameNode sert à…

A) Stocker les blocs de données

B) Conserver l'annuaire des fichiers et l'emplacement des blocs

C) Exécuter les calculs Map

7. L'avantage décisif de Spark par rapport à MapReduce classique est…

A) Il supprime le besoin de stocker les données

B) Il écrit tous ses résultats sur disque

C) Il conserve les données intermédiaires en mémoire

8. Dans un schéma en étoile, la table centrale contient…

A) Les faits : les événements mesurés (ventes, montants, quantités)

B) Les dimensions uniquement

C) Les utilisateurs et leurs mots de passe

9. Un lac de données se distingue d'un entrepôt parce qu'il…

A) Interdit les données brutes

B) Ne contient que des données nettoyées

C) Conserve les données brutes, structurées à la lecture

10. Pour la comptabilité d'une PME (petits volumes, exactitude absolue des transactions), le choix recommandé est…

A) Une base clés-valeurs

B) Une base relationnelle ACID

C) Un lac de données

Corrigés du QCM

1. Réponse B — La Véracité renvoie aux données bruitées, incomplètes ou contradictoires qu'il faut traiter avec prudence.

2. Réponse C — Scaler horizontalement = répartir données et calcul sur davantage de machines ordinaires, au lieu d'une machine unique toujours plus grosse.

3. Réponse B — Le document est l'unité d'enregistrement ; la collection joue le rôle proche de la table, sans schéma rigide.

4. Réponse B — CAP : lors d'une partition, impossible de garantir à la fois que chaque lecture voit la dernière écriture (C) et que chaque requête obtient une réponse (A).

5. Réponse C — BASE = Basically Available, Soft state, Eventually consistent : l'opposé d'esprit des garanties ACID.

6. Réponse B — Le NameNode garde les métadonnées ; les DataNodes stockent les blocs répliqués.

7. Réponse C — Le traitement en mémoire évite les écritures disque entre étapes : les calculs itératifs deviennent beaucoup plus rapides.

8. Réponse A — La table des faits porte les mesures ; les dimensions (temps, produit, magasin…) l'entourent comme les branches d'une étoile.

9. Réponse C — Lac = schéma à la lecture, formats d'origine conservés ; entrepôt = schéma à l'écriture, données nettoyées d'avance.

10. Réponse B — Faible volume, structure stable, cohérence immédiate non négociable : le relationnel est fait exactement pour cela.

Résumé — les notions importantes

• Le Big Data se définit par cinq V : Volume, Vélocité, Variété, Véracité, Valeur ; le relationnel y atteint ses limites en volume, en structure rigide et en disponibilité mondiale.

• Les bases NoSQL se répartissent en familles — clés-valeurs, documents, familles de colonnes, graphes — chacune optimisée pour un type d'accès.

• La distribution repose sur la réplication (copies contre les pannes) et le sharding (répartition des données entre machines).

• CAP : pendant une partition réseau, un système choisit entre cohérence immédiate et disponibilité ; ACID vise la première, BASE (cohérence éventuelle) assume le second choix.

• MongoDB se modélise à partir des requêtes : imbriquer ce qui est lu ensemble et borné (limite 16 Mo), référencer ce qui est partagé ou illimité ; requêtes par find et pipelines $match/$group/$sort.

• Hadoop fournit le socle distribué : HDFS (blocs répliqués, NameNode/DataNodes), MapReduce (Map, shuffle, Reduce) et YARN (ressources) ; Spark accélère les calculs en gardant les étapes intermédiaires en mémoire.

• Entrepôt (données propres, schéma à l'écriture, étoile faits/dimensions) et lac (données brutes, schéma à la lecture) sont complémentaires ; le choix SQL/NoSQL se justifie cas par cas par la structure, les requêtes dominantes et l'exigence de cohérence.

Fiches de révision express

Chaque fiche reprend les points « À retenir » d'une leçon. Relis-les la veille d'un contrôle : cinq minutes par fiche suffisent.

Fiche 1 — Les cinq V du Big Data et les limites du modèle relationnel
Les cinq V : Volume, Vélocité, Variété, Véracité, Valeur — les citer et les illustrer fait partie des questions classiques d'examen. ; Montée en charge verticale = machine plus puissante ; horizontale = plus de machines. Le Big Data repose sur la seconde. ; NoSQL signifie « Not Only SQL » : un complément du relationnel, pas son remplacement systématique.

Fiche 2 — Bases NoSQL : documents, clés-valeurs et familles de colonnes
Clés-valeurs = accès par clé uniquement ; documents = objets JSON complets ; colonnes = écritures massives ; graphes = relations. ; CAP, énoncé exact : pendant une partition réseau, impossible de garantir à la fois cohérence immédiate ET disponibilité — il faut choisir. ; ACID = garanties fortes par transaction ; BASE = disponibilité d'abord, cohérence obtenue « à terme » (éventuelle).

Fiche 3 — Modélisation NoSQL avec MongoDB
On modélise à partir des accès de l'application : les données lues ensemble se stockent ensemble. ; Imbrication = lecture rapide mais duplication ; référence = pas de duplication mais lectures multiples. Il faut arbitrer, pas appliquer une règle aveugle. ; Pipeline d'agrégation : $match filtre, $group regroupe et calcule, $sort trie — l'héritier du GROUP BY de SQL.

Fiche 4 — Traitement distribué : concepts de Hadoop et de Spark
HDFS = stockage distribué en blocs répliqués (3 fois par défaut) ; NameNode = l'annuaire, DataNodes = les blocs. ; MapReduce : Map produit des paires (clé, valeur), le shuffle regroupe par clé, Reduce combine. Simple et robuste, mais avec écritures disque entre étapes. ; Spark = calcul en mémoire sur RDD/DataFrames : plus rapide, notamment pour les traitements itératifs ; il utilise souvent HDFS et YARN d'Hadoop.

Fiche 5 — Entrepôts de données et lacs de données
Schéma en étoile : une table des faits (mesures) entourée de tables de dimensions (axes d'analyse). ; Entrepôt = schéma à l'écriture (propre, décidé d'avance) ; lac = schéma à la lecture (brut, décidé à l'usage). ; Un lac sans catalogue ni règles de qualité devient un marécage de données inutilisable.

Fiche 6 — Choisir entre SQL et NoSQL selon le besoin métier
La cohérence immédiate non négociable (argent, stocks critiques) désigne le relationnel ACID presque toujours. ; Les trois requêtes les plus fréquentes de l'application sont le meilleur test d'un modèle de données. ; Persistance polyglotte : plusieurs bases dans un même système, chacune pour l'usage auquel elle excelle — un choix d'architecture, pas une incohérence.

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 1 — Maîtriser le Big Data Guide Complet pour Ingénieurs et Étudiants

Chaîne : KRIT Academy · Langue : Français · Niveau : Intermédiaire

Une vue d'ensemble complète du Big Data : les cinq V, l'architecture générale, puis les technologies essentielles (Hadoop, Spark, NoSQL). Parfait pour situer toutes les notions du chapitre avant de les détailler leçon par leçon.

https://www.youtube.com/watch?v=eLkqdrFn5jo

Leçon 2 — Utilisez les bases de données NoSQL

Chaîne : OpenClassrooms en Français · Langue : Français · Niveau : Débutant

La vidéo d'introduction du cours OpenClassrooms consacré au NoSQL : pourquoi ces bases existent, ce qu'elles changent par rapport au relationnel et comment MongoDB sert de point d'entrée. Idéale en premier visionnage de la leçon 2.

https://www.youtube.com/watch?v=4RztOLH7fGg

Leçon 3 — MongoDB : documents, collections et requêtes expliqués

Chaîne : Bitdyne · Langue : Français · Niveau : Débutant à intermédiaire

Documents JSON, collections et requêtes find expliqués pas à pas avec du code réellement exécuté, y compris le dilemme imbriquer ou référencer. Le compagnon direct de la leçon 3 et du TP.

https://www.youtube.com/watch?v=hVukp5N78EY

Leçon 4 — Hadoop c'est quoi - Hadoop Tuto Vidéo

Chaîne : Alphorm · Langue : Français · Niveau : Débutant

Une présentation claire d'Apache Hadoop et de son système de fichiers distribué HDFS : à quoi ça sert, comment les données sont réparties et pourquoi les pannes y sont considérées comme normales. À regarder avant d'attaquer MapReduce.

https://www.youtube.com/watch?v=zJZBXFajnyw

Leçon 4 — Hadoop vs Spark : Quelle Technologie Big Data Choisir ? La Vérité Expliquée Simplement

Chaîne : Houssem Lahiani · Langue : Français · Niveau : Débutant à intermédiaire

Le comparatif Hadoop/Spark expliqué simplement : rôle de HDFS et de MapReduce, pourquoi le traitement en mémoire de Spark est plus rapide, et dans quels cas les deux se complètent. Une bonne synthèse de fin de leçon 4.

https://www.youtube.com/watch?v=2k8P3MI0MPs

Sources et références

• Programme officiel Licence Informatique, Université IAM (Institut Africain de Management), Bamako — vérifier les intitulés d'UE auprès du secrétariat

pédagogique.

• Documentation officielle MongoDB (mongodb.com/docs) — manuels sur la modélisation de données, les requêtes et le cadre d'agrégation.

• Documentation Apache Hadoop (hadoop.apache.org) et Apache Spark (spark.apache.org) — guides HDFS, MapReduce, YARN et programmation Spark.

• Brewer, E. — travaux sur le théorème CAP (2000) et son article de clarification « CAP Twelve Years Later » (IEEE Computer, 2012) : la référence pour ne pas caricaturer le théorème.

• Lacomme, P., Aridhi, S. & Phan, R. — « Bases de données NoSQL et Big Data » (Ellipses, Technosup) : ouvrage français avec travaux pratiques sur MongoDB, Cassandra et HBase.

• Les vidéos listées dans la section « Vidéos associées aux leçons » de ce document (liens vérifiés via YouTube à la publication).

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !

Mes notes

Utilise ces pages pour recopier les définitions, refaire les calculs et noter tes questions pour le prochain cours.